

MERN Security Handbook (No-Nonsense, 2025 Edition)

Reality check: 100% security doesn't exist. But you can get *extremely* hard to break. This handbook gives you practical steps, code, and tools to raise the bar dramatically for a MERN app.

0) Threat Model (Decide What You're Defending Against)

- **Assets:** user data (PII), credentials, sessions/JWTs, payments, admin endpoints, source code, secrets, infra.
- **Adversaries:** script kiddies, botnets, opportunistic scanners, targeted attackers, insiders.
- **Attack surfaces:** web/API, WebSockets, file uploads, auth flows, DB queries, CI/CD, supply chain (npm), infra (Docker/K8s), mobile clients.

Write 3–5 one-liners like: *“Protect prod DB from internet; only VPC access; read-only replicas for analytics.”*

1) Golden Rules

1. **Least privilege everywhere.** Give the smallest possible rights (DB users, IAM roles, app roles).
 2. **Deny by default.** Routes, CORS, firewall rules.
 3. **Never trust input.** Validate, sanitize, encode.
 4. **Keep secrets out of code and logs.** Use a secret manager.
 5. **Patch continuously.** Automate dependency updates and audits.
 6. **Defense in depth.** Multiple layers so one miss isn't fatal.
 7. **Secure by default.** Safe defaults in configs and code.
-

2) Core Libraries (Node/Express)

- **HTTP hardening:** helmet, compression, hpp, express-rate-limit, express-slow-down (optional)
- **CORS:** cors (whitelist exact origins)
- **Input validation:** zod (or joi/celebrate) + custom middleware
- **Sanitization:** express-mongo-sanitize (NoSQL injection), validator
- **Auth & crypto:** argon2 (or bcrypt), jose (JWT), crypto (Node core), otplib or speakeasy (TOTP 2FA)
- **Sessions (if cookie-based):** express-session + connect-mongo

- **Cookies:** cookie-parser (read-only), set HttpOnly, Secure, SameSite=Strict
 - **Uploads:** multer, file-type (MIME sniffing), optional antivirus bridge (e.g., ClamAV)
 - **Logging:** pino (fast; redaction), pino-http
 - **Rate-limited login:** rate-limiter-flexible (Redis backend)
 - **WebSockets:** socket.io (with JWT/Session auth, rate-limit per socket)
 - **Security scanning:** npm audit, snyk, semgrep (SAST)
-

3) Express Security Starter (Copy/Paste)

```
import express from 'express';
import helmet from 'helmet';
import compression from 'compression';
import cors from 'cors';
import rateLimit from 'express-rate-limit';
import slowDown from 'express-slow-down';
import hpp from 'hpp';
import mongoSanitize from 'express-mongo-sanitize';
import cookieParser from 'cookie-parser';
import { z } from 'zod';

const app = express();
app.set('trust proxy', 1); // if behind proxy

// Content Security Policy (tighten per app)
app.use(helmet());

// Optional fine-grained CSP example:
// app.use(helmet.contentSecurityPolicy({
//   useDefaults: true,
//   directives: {
//     "default-src": ["'self'"],
//     "script-src": ["'self'", "'nonce-<generate-per-request>'"],
//     "object-src": ["'none'"],
//     "base-uri": ["'self'"],
//     "frame-ancestors": ["'none'"],
//   },
// }));

app.use(compression());
app.use(express.json({ limit: '200kb' }));
app.use(express.urlencoded({ extended: false, limit: '200kb' }));
app.use(cookieParser());
app.use(hpp());
app.use(mongoSanitize());

// CORS: strict allow-list
```

```

const allowed = new Set([process.env.CLIENT_URL]);
app.use(cors({
  origin: (origin, cb) => {
    if (!origin || allowed.has(origin)) return cb(null, true);
    cb(new Error('Not allowed by CORS'));
  },
  credentials: true,
}));

// Global rate limit + slowdown
const limiter = rateLimit({ windowMs: 15 * 60 * 1000, max: 300 });
const speed = slowDown({ windowMs: 60 * 1000, delayAfter: 100, delayMs: 500 });
app.use(limiter);
app.use(speed);

// Example validated route
const LoginSchema = z.object({
  username: z.string().trim().min(3).max(64),
  password: z.string().min(8).max(128),
});
app.post('/api/v1/auth/login', (req, res, next) => {
  const parsed = LoginSchema.safeParse(req.body);
  if (!parsed.success) return res.status(400).json({ message: 'Invalid input' });
  return next();
});

// Error handler (no leaks)
app.use((err, req, res, _next) => {
  req.log?.error?.(err);
  return res.status(500).json({ message: 'Something went wrong' });
});

export default app;

```

4) Auth & Sessions (JWT vs Cookies)

- **Prefer HttpOnly Secure cookies** for browser clients. Mark SameSite=Strict, set short TTL, rotate on login/refresh.
- **JWT tips (if used):**
 - Sign with `` using RS256/EdDSA and rotate keys.
 - Keep access token ~5–15 min; use refresh token (rotating) in HttpOnly cookie.
 - Include aud, iss, exp, iat, jti. Validate all.
 - **Do not** store JWTs in localStorage for sensitive apps.

- **Passwords:** argon2id (or bcrypt cost ≥ 12). Use zxcvbn for strength feedback.
- **2FA (TOTP):** speakeasy/otplib + backup codes. Short step-by-step onboarding.
- **Password resets:** random 32B tokens, hashed in DB, TTL 10–15 min, single-use, invalidate all sessions on change.

JWT verification snippet (jose):

```
import * as jose from 'jose';
const JWKS = jose.createLocalJWKSet({ keys: [/* your rotated keys */] });

export async function verifyJWT(token, expectedAud, expectedIss) {
  const { payload } = await jose.jwtVerify(token, JWKS, {
    algorithms: ['RS256', 'EdDSA'],
    audience: expectedAud,
    issuer: expectedIss,
  });
  return payload;
}
```

5) MongoDB Security (NoSQL Injection & Config)

- **Config:**
 - Enable **auth** with SCRAM-SHA-256.
 - Bind to **localhost/VPC only** (no public internet).
 - Enforce **TLS** on client connections.
 - Create separate DB users per environment with least privilege.
 - Disable/avoid \$where & server-side JS.
- **App-layer:**
 - Use express-mongo-sanitize to strip \$/. in payloads.
 - Never concatenate into query operators from user input; map inputs to **whitelisted fields**.
 - Validate schemas at Mongoose level (required, enum, minlength, maxlength).
 - Set query timeouts/limits (maxTimeMS) for long-running queries.
 - Paginate results; never return unbounded arrays.

Example query allow-list:

```
const SORT_FIELDS = new Set(['createdAt', 'price', 'name']);
const sort = req.query.sort;
const sortBy = SORT_FIELDS.has(sort) ? sort : 'createdAt';
Model.find(filter).sort({ [sortBy]: 1 }).limit(50);
```

6) XSS, CSRF, Clickjacking

- **XSS:**
 - **React** escapes by default, but any HTML rendering must use **DOMPurify** on the client.
 - Set **CSP** with nonces/hashes; disable inline scripts.
 - Encode all output; never echo raw user content.
 - **CSRF:**
 - If using cookies, add ``. For SPA + cookie JWT, pair CSRF token (double submit) or use **SameSite=Strict** and check Origin/Referer on sensitive routes.
 - **Clickjacking:** `helmet.frameguard({ action: 'deny' })`.
 - **MIME sniffing:** `helmet.noSniff()`.
-

7) File Upload Safety

- Accept-list mimetypes; verify with file-type (don't trust Content-Type).
 - Enforce size limits, dimensions (for images), and count.
 - Store outside web root; prefer object storage (S3/GCS) with pre-signed URLs.
 - Strip metadata; optionally run antivirus (ClamAV) for user files.
 - Transform images server-side (sharp) to safe formats (e.g., PNG/JPEG) to kill payloads.
-

8) WebSockets (Socket.IO) Hardening

- Authenticate on **connection** (JWT/cookie session).
 - Check **origin**; deny unauthorised domains.
 - Limit **events/sec**, message **size**, and **room joins**.
 - Authorize per-event; never trust client-sent `userId`.
 - Disconnect on abuse; IP + user rate limits.
-

9) Secrets Management

- Do **not** commit `.env` to VCS. Use `.env.example` only.
 - Use a **secret manager** (Vault, AWS/GCP/Azure) in prod. Rotate keys.
 - Provide secrets via env vars at runtime; mount with least privilege.
 - Redact secrets in logs; verify with `pino` redaction config.
-

10) Secure Deployment (Docker/K8s/Servers)

- Minimal base images (node:18-alpine or distroless), **non-root** user, read-only FS where possible.
 - Drop Linux capabilities; use seccomp/apparmor.
 - Expose only needed ports; put **reverse proxy** (Nginx/Caddy) in front with TLS 1.2+ and **HSTS**.
 - Healthchecks, liveness/readiness probes; auto-restarts.
 - Backups encrypted; test restores regularly.
 - Logs shipped to central store with alerting (e.g., Loki/ELK) and **WAF** in front if public.
-

11) CI/CD & Supply Chain

- Lock deps (package-lock.json), npm ci in CI.
 - **Dependabot/Renovate** + **Snyk** + npm audit in pipeline; block on critical vulns.
 - **Semgrep** (SAST) and **OWASP ZAP/Burp** (DAST) in staging.
 - Sign builds/images (Sigstore/Cosign). Verify at deploy.
 - Pin actions in GitHub Actions (@v3 with commit SHA).
-

12) Logging, Audit, Monitoring

- Structured logs (pino), request IDs, user IDs (server-derived), and important events.
 - Audit trails for auth, admin actions, data exports/deletes.
 - Alerting for spikes: 401/403, 5xx, login failures, rate-limit hits.
 - Keep metrics (Prometheus/Grafana). Create runbooks for incidents.
-

13) Frontend (React) Security

- Avoid dangerouslySetInnerHTML; if necessary, sanitize with **DOMPurify**.
- Use **CSP nonces**; avoid inline scripts/styles.
- Don't store secrets/tokens in the client; if unavoidable, scope and expire tightly.
- Validate everything again on the server (never trust the client).
- Handle errors generically; don't leak stack traces.

DOMPurify example:

```
import DOMPurify from 'dompurify';
const clean = DOMPurify.sanitize(untrustedHtml, { USE_PROFILES: { html: true } });
```

14) Practical Checklists

Pre-commit

Pre-deploy (staging)

Production

15) Example Security Middleware Bundle

```
export function securityBundle(app) {  
  ap
```